

МРС-65 — Реализовать явные режимы и команды запуска Worker

Summary

Перевести `VarPrice.Worker` с неявного или частично управляемого запуска на строгую command-based модель.

После выполнения задачи Worker должен поддерживать независимые режимы:

```
catalog-refresh  
collect-prices  
run-all  
help
```

Каждая команда должна запускать только предназначенный для неё application use case.

Worker не должен:

- угадывать режим;
- молча запускать legacy crawler при ошибочной команде;
- выполнять discovery при `collect-prices`;
- выполнять price collection при `catalog-refresh`;
- смешивать orchestration и бизнес-логику в `Program.cs`;
- возвращать exit code `0` при фактической ошибке запуска.

Главная цель:

```
Worker становится предсказуемым CLI entry point,  
пригодным для ручного запуска, cron и cloud scheduler.
```

Context

Репозиторий:

```
https://github.com/bodomus/PriceCrawler
```

Базовая ветка:

```
МРС-64
```

Создать рабочую ветку:

Codex/MPC-65

Dependencies

Задача зависит от завершённых:

MPC-61 – product_catalog
MPC-62 – RefreshProductCatalogUseCase
MPC-63 – CollectProductPricesUseCase
MPC-64 – catalog deactivation lifecycle

До начала реализации убедиться, что существуют:

IRefreshProductCatalogUseCase
ICollectProductPricesUseCase

и Worker уже способен разрешать их через DI.

Если один из use case отсутствует, не создавать временную дублирующую бизнес-логику внутри Worker. Зафиксировать блокирующую зависимость.

Problem

Текущий Worker исторически развивался вокруг одного crawler flow.

После появления отдельных сценариев:

catalog refresh
price collection

необходимо явно разделить способы запуска.

Без строгой command model возникают риски:

- случайный запуск обхода всех категорий;
- запуск не того use case из scheduler;
- невозможность однозначно интерпретировать exit code;
- сложность автоматизации;
- скрытый fallback на legacy flow;
- смешение catalog refresh и price collection;
- ошибки конфигурации обнаруживаются слишком поздно;

- невозможно безопасно запускать Worker из CI/CD или cron.
-

Goal

Реализовать CLI flow:

```
VarPrice.Worker <command> [options]
```

Обязательные команды:

```
catalog-refresh  
collect-prices  
run-all  
help
```

Допустимые aliases:

```
--help  
-h
```

Команда без аргументов должна иметь строго определённое поведение и не должна молча запускать crawler.

Required behavior overview

`catalog-refresh`

Запускает только:

```
IReshProductCatalogUseCase.ExecuteAsync(...)
```

Не запускает:

- `ICollectProductPricesUseCase` ;
 - legacy `RunCrawlerUseCase` ;
 - price queue drain отдельно;
 - snapshot collection;
 - `run-all` .
-

collect-prices

Запускает только:

```
ICollectProductPricesUseCase.ExecuteAsync(...)
```

Не запускает:

- discovery;
 - catalog refresh;
 - legacy `RunCrawlerUseCase` ;
 - category page loading;
 - sitemap discovery.
-

run-all

Последовательно выполняет:

```
catalog-refresh  
→ collect-prices
```

`collect-prices` запускается только если catalog refresh завершился успешно.

При ошибке catalog refresh:

```
collect-prices не запускается  
process exit code != 0
```

help

Выводит:

- список команд;
- краткое назначение каждой;
- примеры запуска;
- возможные exit codes;
- текущее правило запуска без аргументов.

После `help`:

```
exit code = 0
```

Architecture requirements

Соблюдать слои:

```
Domain
Application
Infrastructure
Worker
Web
Tests
```

Worker responsibilities

Worker отвечает только за:

- разбор аргументов;
- выбор команды;
- создание DI scope;
- разрешение use case;
- передачу `CancellationToken`;
- обработку результата;
- формирование exit code;
- агрегированное CLI-логирование.

Worker не должен содержать:

- SQL;
- product selection;
- oldest-first logic;
- catalog deactivation logic;
- URL filtering;
- queue retry policy;
- HTTP extraction logic;
- snapshot persistence;
- discovery orchestration.

Scope

1. Добавить command abstraction

Добавить application-agnostic Worker contract:

```
public interface IWorkerCommand
{
    string Name { get; }
```

```
Task<WorkerCommandResult> ExecuteAsync(  
    CancellationToken ct);  
}
```

Допускается расположить интерфейс в `Worker`, поскольку это CLI-specific contract.

Не помещать его в `Domain`.

2. Добавить `WorkerCommandResult`

Добавить immutable result:

```
public sealed record WorkerCommandResult(  
    string Command,  
    bool Success,  
    int ExitCode,  
    long? RunId,  
    string? ErrorCode,  
    string Message);
```

Требования

`ExitCode` должен соответствовать единой таблице кодов.

`Success = true` допустим только при:

```
ExitCode = 0
```

`Success = false` не должен возвращать `ExitCode = 0`.

`Message` не должен содержать stack trace.

3. Добавить команды как отдельные классы

Добавить:

```
CatalogRefreshWorkerCommand  
CollectPricesWorkerCommand  
RunAllWorkerCommand  
HelpWorkerCommand
```

Предпочтительное расположение:

```
VarPrice.Worker/Commands
```

Каждая команда должна быть отдельным классом.

Не реализовывать весь CLI через один большой `switch` с бизнес-логикой внутри `Program.cs`.

CatalogRefreshWorkerCommand

4. Реализация

Класс должен зависеть от:

```
IReloadProductCatalogUseCase  
ILogger<CatalogRefreshWorkerCommand>
```

Flow:

```
лог начала  
→ ExecuteAsync  
→ анализ результата  
→ формирование WorkerCommandResult  
→ лог завершения
```

Успех

Если use case вернул:

```
Status = ok
```

то:

```
Success = true  
ExitCode = 0  
RunId = result.RunId
```

Ошибка

Если use case вернул:

```
Status = error
```

то:

```
Success = false  
ExitCode = соответствующий non-zero code  
RunId = result.RunId  
ErrorCode = result.ErrorCode
```

Команда не должна повторно выполнять catalog refresh.

CollectPricesWorkerCommand

5. Реализация

Класс должен зависеть от:

```
ICollectProductPricesUseCase  
ILogger<CollectPricesWorkerCommand>
```

Успех

```
Status = ok  
→ ExitCode = 0
```

Пустой due batch:

```
SelectedCount = 0  
Status = ok
```

считается успешным запуском.

Ошибка

Если:

```
DeadCount > 0  
Status = error
```

то команда возвращает ненулевой exit code.

RunAllWorkerCommand

6. Реализация

Класс должен зависеть от:

```
IRefreshProductCatalogUseCase  
ICollectProductPricesUseCase  
ILogger<RunAllWorkerCommand>
```

или переиспользовать:

```
CatalogRefreshWorkerCommand  
CollectPricesWorkerCommand
```

Предпочтительно переиспользовать command abstractions, если это не создаёт проблем с DI scopes.

Required flow

```
catalog refresh start  
→ catalog refresh finish  
→ if success:  
    price collection start  
    price collection finish  
else:  
    stop
```

Запрещено

Не запускать обе операции параллельно.

Не запускать price collection в `finally`.

Не игнорировать ошибку catalog refresh.

7. Run-all result

При полном успехе:

```
Success = true  
ExitCode = 0
```

```
RunId = RunId последнего этапа либо null
Message содержит агрегированный результат обоих этапов
```

При ошибке catalog refresh:

```
Success = false
collect-prices не вызван
ExitCode != 0
ErrorCode = error code catalog refresh
```

При успешном catalog refresh и ошибке price collection:

```
Success = false
ExitCode != 0
ErrorCode = error code price collection
```

Command parser

8. Добавить parser

Добавить отдельный компонент:

```
IWorkerCommandParser
WorkerCommandParser
```

или stateless equivalent.

Пример результата:

```
public sealed record WorkerCommandRequest(
    string CommandName,
    bool ShowHelp);
```

Допускается более простой тип, если он полностью тестируется.

9. Нормализация аргументов

Команды должны распознаваться:

- без учёта регистра;
- после `Trim`;

- только как точное имя команды.

Допустимо:

```
catalog-refresh  
CATALOG-REFRESH  
Catalog-Refresh
```

Недопустимо:

```
catalog  
refresh  
catalog-refresh-extra
```

10. Поведение без аргументов

Обязательное поведение:

```
аргументы отсутствуют  
→ вывести help  
→ exit code = 2  
→ никакой use case не запускается
```

Не сохранять старый неявный запуск crawler без аргументов.

Это намеренное breaking change CLI behavior, необходимое для безопасности.

README должен явно это описывать.

11. Неизвестная команда

Пример:

```
dotnet run --project VarPrice.Worker -- unknown
```

Поведение:

```
вывести ошибку  
вывести краткую help-подсказку  
не запускать use case  
exit code = 2
```

Не выполнять fallback.

12. Лишние аргументы

Для MPC-65 команды пока не принимают command-specific options.

Пример:

```
collect-prices --foo
```

Поведение:

```
ошибка CLI  
exit code = 2  
use case не запускается
```

Не игнорировать неизвестные аргументы.

Исключения:

```
help  
--help  
-h
```

Exit codes

13. Добавить централизованные константы

Добавить:

```
public static class WorkerExitCodes  
{  
    public const int Success = 0;  
    public const int CommandError = 2;  
    public const int ConfigurationError = 3;  
    public const int CatalogRefreshFailed = 10;  
    public const int PriceCollectionFailed = 11;  
    public const int Cancelled = 12;  
    public const int UnexpectedError = 20;  
}
```

Допускается другой набор значений, но он должен быть:

- централизован;
- документирован;
- покрыт тестами;
- стабилен.

14. Mapping ошибок

Success

0

Missing/invalid command

2

Configuration startup failure

3

Catalog refresh application failure

10

Price collection application failure

11

Cancellation

12

Unexpected unhandled error

20

Не возвращать платформенно-зависимые случайные коды.

Configuration startup errors

15. Ошибки построения Host

Если приложение не может стартовать из-за:

- отсутствующего connection string;
- отсутствующего config file;
- invalid JSON;
- invalid options;
- DI resolution failure;

Worker должен:

- вывести краткую ошибку;
- вернуть `ConfigurationError` либо `UnexpectedError`;
- не выводить секреты;
- не печатать пароль из connection string;
- не создавать ложный successful run.

Если ошибка произошла до запуска logging infrastructure, допустим `Console.Error`.

Cancellation and process signals

16. Ctrl+C

Worker должен корректно использовать:

```
Console.CancelKeyPress
```

или стандартный:

```
IHostApplicationLifetime.ApplicationStopping
```

Передавать cancellation token в command/use case.

При отмене:

- не запускать следующий этап `run-all`;
 - не возвращать exit code `0`;
 - использовать `WorkerExitCodes.Cancelled`;
 - не поглощать `OperationCanceledException` как обычный success;
 - дать application use case выполнить предусмотренный cleanup.
-

17. Повторный Ctrl+C

Не добавлять сложную signal handling систему.

Допускается стандартное немедленное завершение процесса при повторном сигнале.

Program.cs

18. Требования к Program.cs

Program.cs должен оставаться компактным.

Предпочтительная структура:

```
parse arguments
→ build host
→ create scope
→ resolve command dispatcher
→ execute
→ return exit code
```

Не помещать в Program.cs :

- RefreshProductCatalogUseCase orchestration;
 - CollectProductPricesUseCase orchestration;
 - queue processing;
 - repository calls;
 - SQL;
 - длинный try/catch на каждый сценарий;
 - форматирование всех command messages.
-

Command dispatcher

19. Добавить dispatcher

Добавить:

```
public interface IWorkerCommandDispatcher
{
    Task<WorkerCommandResult> DispatchAsync(
        IReadOnlyList<string> args,
```

```
        CancellationToken ct);  
    }
```

Реализация должна:

1. разобрать аргументы;
2. определить command;
3. разрешить соответствующую реализацию;
4. выполнить её;
5. вернуть единый result.

20. Command registry

Предпочтительно зарегистрировать команды через DI:

```
services.AddScoped<IWorkerCommand, CatalogRefreshWorkerCommand>();  
services.AddScoped<IWorkerCommand, CollectPricesWorkerCommand>();  
services.AddScoped<IWorkerCommand, RunAllWorkerCommand>();  
services.AddScoped<IWorkerCommand, HelpWorkerCommand>();
```

Dispatcher может построить словарь:

```
Name → IWorkerCommand
```

Проверить отсутствие duplicate command names.

При duplicate names:

- startup/dispatch должен завершиться ошибкой;
- не выбирать команду случайным образом.

DI lifetimes

21. Scope requirements

Каждый Worker process command должен выполняться в одном DI scope.

Для `run-all` предпочтительно:

```
один command scope
```

Но каждый application use case должен получать корректные scoped dependencies.

Не использовать singleton для:

- repository;
 - DbContext;
 - PgRoutineExecutor;
 - use case;
 - command, содержащей scoped dependencies.
-

Legacy flow

22. Старый RunCrawlerUseCase

После MPC-65 legacy use case может оставаться в коде, но не должен быть доступен как неявный default command.

Допустимы два варианта:

Предпочтительно

Legacy flow не регистрируется как CLI command.

Допустимо

Добавить явную диагностическую команду:

```
legacy-crawl
```

только если она нужна существующим тестам или ручной диагностике.

Если добавлена:

- она должна быть явно помечена как legacy;
- не упоминаться как production mode;
- не запускаться без аргументов;
- иметь отдельные тесты.

`legacy-crawl` не является обязательной частью MPC-65.

Output format

23. Console output

Каждая команда должна выводить финальный агрегированный summary.

Catalog refresh

Пример:

```
Command: catalog-refresh  
Status: ok  
RunId: 125  
RefreshId: 18  
Discovered: 40120  
Inserted: 150  
Updated: 39950  
Reactivated: 20  
Deactivated: 5
```

Collect prices

Пример:

```
Command: collect-prices  
Status: ok  
RunId: 126  
Selected: 5000  
Enqueued: 5000  
Succeeded: 4930  
Retry: 40  
Dead: 30
```

Run-all

Пример:

```
Command: run-all  
Catalog refresh: ok  
Price collection: ok  
Status: ok
```

Не выводить:

- все URL;
 - connection string;
 - пароли;
 - stack trace при обычной ожидаемой ошибке;
 - полные JSON batches.
-

24. stdout / stderr

Использовать:

```
stdout
```

для:

- help;
- successful summaries;
- normal informational output.

Использовать:

```
stderr
```

для:

- invalid command;
- configuration error;
- failed command;
- unexpected error.

Structured logs могут продолжать идти через Serilog.

Logging

25. Required logs

Dispatch

```
Worker command dispatch started. Command={Command}
```

Completion

```
Worker command completed. Command={Command}; Success={Success};  
ExitCode={ExitCode}; RunId={RunId}
```

Error

```
Worker command failed. Command={Command}; ExitCode={ExitCode};  
ErrorCode={ErrorCode}
```

Исключение передавать logger параметром.

Не логировать полный массив args, если в будущем там могут появиться secrets.

Допускается логировать только command name.

Help text

26. Обязательное содержание

Help должен содержать:

```
Usage:
  VarPrice.Worker <command>

Commands:
  catalog-refresh  Discover and synchronize the Varus product catalog
  collect-prices   Collect prices for due catalog products
  run-all         Refresh catalog, then collect prices
  help            Show this help

Aliases:
  -h, --help

Exit codes:
  0   Success
  2   Invalid or missing command
  3   Configuration/startup error
  10  Catalog refresh failed
  11  Price collection failed
  12  Cancelled
  20  Unexpected error
```

Можно добавить русское описание в README, но CLI help желательно оставить на одном языке. Предпочтительно английский для технической среды.

Tests

Добавить unit tests для parser, dispatcher и каждой команды.

Parser tests

1. Catalog refresh

```
Parse_CatalogRefresh_ReturnsCatalogRefreshCommand
```

2. Collect prices

```
Parse_CollectPrices_ReturnsCollectPricesCommand
```

3. Run all

```
Parse_RunAll_ReturnsRunAllCommand
```

4. Help

```
Parse_Help_ReturnsHelpCommand
```

5. Help aliases

```
Parse_HelpAliases_ReturnHelpCommand
```

Проверить:

```
-h  
--help
```

6. Case insensitive

```
Parse_Command_IsCaseInsensitive
```

7. No arguments

```
Parse_NoArguments_ReturnsCommandError
```

8. Unknown command

```
Parse_UnknownCommand_ReturnsCommandError
```

9. Extra arguments

Parse_ExtraArguments_ReturnsCommandError

Dispatcher tests

10. Correct command selected

DispatchAsync_CatalogRefresh_ExecutesOnlyCatalogCommand

11. Collect command isolated

DispatchAsync_CollectPrices_ExecutesOnlyCollectCommand

12. Unknown command executes nothing

DispatchAsync_UnknownCommand_DoesNotExecuteAnyUseCase

13. Duplicate command registration

Dispatcher_DuplicateCommandNames_ThrowsConfigurationError

Catalog command tests

14. Success

CatalogRefreshCommand_UseCaseOk_ReturnsExitCodeZero

15. Application error

CatalogRefreshCommand_UseCaseError_ReturnsCatalogFailureExitCode

16. Run ID mapping

```
CatalogRefreshCommand_MapsRunIdAndErrorCode
```

17. Cancellation

```
CatalogRefreshCommand_Cancelled_ReturnsCancelledExitCode
```

или повторно выбрасывает cancellation до верхнего command boundary, где он маппится в код 12.

Выбрать один единый подход.

Collect command tests

18. Success

```
CollectPricesCommand_UseCaseOk_ReturnsExitCodeZero
```

19. Empty batch success

```
CollectPricesCommand_NoDueProducts_ReturnsExitCodeZero
```

20. Dead items error

```
CollectPricesCommand_UseCaseError_ReturnsPriceCollectionFailureExitCode
```

21. No discovery dependency

Проверить, что command не зависит от discovery service.

Run-all tests

22. Both successful

```
RunAllCommand_BothStagesSuccessful_ReturnsSuccess
```

Порядок:

```
catalog  
collect
```

23. Catalog failure stops pipeline

```
RunAllCommand_CatalogFails_DoesNotRunCollectPrices
```

24. Collect failure

```
RunAllCommand_CollectFails_ReturnsPriceCollectionFailure
```

25. Cancellation after catalog

```
RunAllCommand_CancelledBeforeCollect_DoesNotStartCollect
```

26. Exact invocation count

Каждый use case вызывается максимум один раз.

Program integration tests

27. No args

Запустить Worker без аргументов.

Проверить:

```
exit code = 2  
help displayed  
no crawler_run created
```

28. Help

```
dotnet run --project VarPrice.Worker -- help
```

Проверить:


```
exit code = 0  
no database writes
```

Если host сейчас требует DB даже для help, желательно организовать parsing/help до построения полного host, чтобы help работал без PostgreSQL.

Это предпочтительное требование.

29. Unknown command

```
dotnet run --project VarPrice.Worker -- invalid-command
```

Проверить:

```
exit code = 2  
no use case invoked  
no database writes
```

30. Catalog refresh command

C fake use case либо controlled integration environment:

```
только catalog use case invoked
```

31. Collect prices command

Проверить:

```
только price collection use case invoked
```

32. Run-all order

Проверить фактический порядок через test doubles или controlled integration setup.

Optional framework decision

Допускается использовать:

```
System.CommandLine
```

только если:

- версия пакета стабильна;
- не добавляется preview dependency;
- integration остаётся простой;
- пакет оправдан текущим score;
- все команды покрыты тестами.

Предпочтительно не добавлять внешний CLI framework для четырёх простых команд.

Ручной parser допустим и, вероятно, проще.

README changes

Обновить README.

Добавить раздел

```
Worker commands
```

Примеры

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

```
dotnet run --project VarPrice.Worker -- collect-prices
```

```
dotnet run --project VarPrice.Worker -- run-all
```

```
dotnet run --project VarPrice.Worker -- help
```

Production examples

Еженедельный refresh

```
dotnet VarPrice.Worker.dll catalog-refresh
```

Ежедневный collection

```
dotnet VarPrice.Worker.dll collect-prices
```

Указать

Запуск без команды больше не запускает crawler:

```
no args → help + exit code 2
```

Architecture documentation

Обновить:

```
docs/architecture.md
```

Добавить:

```
Worker CLI
  ↓
Command Dispatcher
  ├── catalog-refresh → RefreshProductCatalogUseCase
  ├── collect-prices  → CollectProductPricesUseCase
  └── run-all          → refresh → collection
```

Явно описать, что Worker:

- не содержит business logic;
- не выполняет repository calls напрямую;
- не выбирает товары;
- не выполняет discovery напрямую;
- является process-level orchestrator.

Non-goals

В рамках MPC-65 запрещено реализовывать:

- cron scheduler;
- Windows Service;
- systemd unit;
- Kubernetes CronJob;
- cloud deployment;
- Docker scheduling;
- interactive menu;
- GUI;
- Web API запуска Worker;
- command options типа `--limit`;
- переопределение connection string через CLI;
- ручное указание RPS через CLI;
- команды миграции БД;
- команды очистки queue;
- команды удаления catalog;
- multi-store commands;
- ATB;
- distributed execution;
- remote command server;
- RabbitMQ;
- Kafka;
- Redis;
- полный observability dashboard;
- изменение бизнес-логики catalog refresh;
- изменение oldest-first selection;
- изменение deactivation policy;
- удаление legacy use case из solution;
- рефакторинг несвязанных проектов.

Important design decisions

1. Команда обязательна

Worker без аргументов не запускает crawler.

2. Нет silent fallback

Неизвестная команда не запускает legacy flow.

3. Use cases изолированы

`catalog-refresh` и `collect-prices` не вызывают друг друга.

4. `run-all` последователен

Price collection начинается только после успешного refresh.

5. Exit code является частью контракта

Scheduler должен однозначно понимать результат запуска.

6. Worker не содержит business logic

CLI orchestration отделена от Application.

7. Help должен быть безопасным

Желательно, чтобы help и invalid command обрабатывались до подключения к БД.

Acceptance criteria

Задача считается выполненной, если:

- [] добавлен command parser;
- [] добавлен command dispatcher;
- [] добавлен `WorkerCommandResult` ;
- [] добавлены централизованные exit codes;
- [] добавлена команда `catalog-refresh` ;
- [] добавлена команда `collect-prices` ;
- [] добавлена команда `run-all` ;
- [] добавлена команда `help` ;
- [] поддерживаются `-h` и `--help` ;
- [] команды распознаются без учёта регистра;
- [] запуск без аргументов возвращает code `2` ;
- [] запуск без аргументов не вызывает use cases;
- [] неизвестная команда возвращает code `2` ;
- [] неизвестная команда не вызывает use cases;
- [] лишние аргументы приводят к command error;
- [] catalog command вызывает только catalog refresh use case;
- [] collect command вызывает только price collection use case;
- [] run-all выполняет use cases последовательно;
- [] при ошибке catalog refresh collect-prices не запускается;
- [] cancellation возвращает стабильный non-zero exit code;
- [] unexpected error возвращает стабильный exit code;
- [] configuration error не выводит secrets;
- [] `Program.cs` не содержит бизнес-логику;
- [] команды зарегистрированы через DI;
- [] duplicate command names обнаруживаются;
- [] финальный summary выводится агрегированно;
- [] help содержит usage, commands и exit codes;

- [] stdout/stderr используются корректно;
- [] добавлены parser tests;
- [] добавлены dispatcher tests;
- [] добавлены command tests;
- [] добавлены integration tests CLI;
- [] README обновлён;
- [] architecture documentation обновлена;
- [] существующие тесты проходят;
- [] solution собирается без новых warnings;
- [] unrelated code не рефакторился.

Required verification

Codex обязан выполнить:

```
dotnet restore
dotnet build --no-restore
dotnet test --no-build
```

Дополнительно проверить exit codes запуском собранного Worker.

Manual verification

1. Help

```
dotnet run --project VarPrice.Worker -- help
```

Ожидается:

```
exit code = 0
```

2. Help alias

```
dotnet run --project VarPrice.Worker -- --help
```

Ожидается:

```
exit code = 0
```

3. No arguments

```
dotnet run --project VarPrice.Worker
```

Ожидается:

```
help output  
exit code = 2  
no crawler run created
```

4. Invalid command

```
dotnet run --project VarPrice.Worker -- invalid
```

Ожидается:

```
exit code = 2  
no use case invoked
```

5. Catalog refresh

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

Проверить:

```
RefreshProductCatalogUseCase invoked once  
CollectProductPricesUseCase not invoked
```

6. Collect prices

```
dotnet run --project VarPrice.Worker -- collect-prices
```

Проверить:

```
CollectProductPricesUseCase invoked once  
RefreshProductCatalogUseCase not invoked
```

7. Run all

```
dotnet run --project VarPrice.Worker -- run-all
```

Проверить порядок:

```
catalog-refresh  
collect-prices
```

8. Catalog failure in run-all

В controlled test environment заставить catalog refresh вернуть error.

Ожидается:

```
collect-prices not invoked  
exit code = 10
```

9. Price collection failure

Заставить collection вернуть error.

Ожидается:

```
exit code = 11
```

10. Cancellation

Запустить долгую команду и нажать:

```
Ctrl+C
```

Ожидается:

process finishes with exit code 12
next run-all stage does not start

SQL verification

После `help`, no args и invalid command:

```
select *  
from crawler_run  
where started_at >= :test_start  
order by id;
```

Ожидается:

0 rows

После `catalog-refresh`:

```
select source, status  
from crawler_run  
order by id desc  
limit 1;
```

Ожидается source catalog refresh flow.

После `collect-prices` ожидается source price collection flow.

Codex final response format

Финальный ответ Codex должен содержать:

1. Summary

Кратко описать реализованную command model.

2. Commands

Привести таблицу:

Command	Use case	Exit code on success
catalog-refresh	RefreshProductCatalogUseCase	0
collect-prices	CollectProductPricesUseCase	0
run-all	Refresh → Collect	0
help	none	0

3. Changed files

Перечислить все изменённые и добавленные файлы с назначением.

4. Dispatch architecture

Показать flow:

```
args
→ parser
→ dispatcher
→ command
→ application use case
→ WorkerCommandResult
→ process exit code
```

5. Exit codes

Перечислить фактические коды и их семантику.

6. Isolation evidence

Подтвердить:

```
catalog-refresh does not call collect-prices
collect-prices does not call discovery/catalog refresh
invalid command calls no use case
```

7. Run-all behavior

Подтвердить:

- последовательный порядок;
- stop-on-failure;
- cancellation behavior.

8. Tests

Указать:

- parser tests;
- dispatcher tests;
- command tests;
- integration tests;
- фактический результат `dotnet build`;
- фактический результат `dotnet test`.

9. Manual verification

Привести фактические результаты:

```
help exit code:
no-args exit code:
invalid-command exit code:
catalog-refresh exit code:
collect-prices exit code:
run-all exit code:
```

10. Risks and follow-up

Явно указать:

- scheduler ещё не реализован;
- command-specific options ещё не поддерживаются;
- legacy use case может оставаться в коде, но не является default mode;
- cloud deployment относится к следующему этапу.

Не заявлять об успешном тестировании или ручной проверке, если команды фактически не запускались.